

DevOps Case Study Report - Nguyen Tran Bao Quoc

0. Tổng quan demo project

Nội dung demo bao gồm:

- Tạo 1 ứng dụng web bằng NodeJS
- Sử dụng Jenkins CI/CD để tự động checkout code, unit test, build image và push lên DockerHub.
- Sử dụng ArgoCD để tự động deploy/rollback app lên môi trường Kubernetes.
- Sử dụng Prometheus, Grafana và Loki để giám sát app và Jenkins controller.
- Sử dụng các công cụ đã cấu hình để xử lý khi app gặp sự cố.

1. Task 0&1 - Kubernetes deployment and Jenkins CI/CD pipeline

1.1. Ứng dụng demo

demo-app là 1 ứng dụng web đơn giản chạy bằng Node và ExpressJS.

```
// Root trả về "Hello from NodeJS web app!" khi truy cập
app.get('/', (req, res) => {
  res.json({
    message: 'Hello from NodeJS web app!',
    hostname: os.hostname(),
    timestamp: new Date().toISOString(),
  });
});

// /healthz trả về "ok" khi truy cập (dùng để healthcheck trong pod)
app.get('/healthz', (req, res) => {
  res.status(200).send('ok');
});
```

1.2. Thiết kế pipeline

Jenkins pipeline bao gồm các stage sau:

- **Checkout:** Tải source code từ repo về agent.
- **Install dependencies:** Cài đặt các package cho app.
- **Unit test:** Chạy các unit test.
- **Docker build:** Đóng gói app thành Docker image. Image được tag theo format **build[build-number]-[commit-SHA]** (vd: build7-635f1c5)
- **Docker push:** Login vào DockerHub và đẩy image lên hub.
- **Update GitOps repo:** Cập nhật image tag trong file values.yaml. ArgoCD trong cluster sẽ tự động cập nhật app.

Environment variables:

Các biến môi trường dùng trong pipeline được khai báo trong block `environment`.

```
environment {  
  REGISTRY      = 'docker.io/tonyq2k3'  
  IMAGE_NAME    = "${REGISTRY}/demo-app"  
  IMAGE_TAG     = "build${env.BUILD_NUMBER}-${env.GIT_COMMIT?.take(7) ?:  
'local'}"  
  CHART_PATH    = 'helm/demo-app'  
  
  GITOPS_REPO   = 'github.com/TonyQ2k3/devops-demo.git'  
  GITOPS_BRANCH = 'dev'  
  VALUES_FILE  = "${CHART_PATH}/values.yaml"  
}
```

Credentials:

Pipeline sử dụng Jenkins Credentials lưu thông tin xác thực DockerHub (để push image) và Github (để update GitOps repo).

Trong pipeline, credential được gọi thông qua `withCredential`:

```
withCredentials([usernamePassword(  
  credentialsId: 'dockerhub-cred',  
  usernameVariable: 'DOCKER_USER',  
  passwordVariable: 'DOCKER_PASS'  
)]) {}  
  
withCredentials([usernamePassword(  
  credentialsId: 'github-pat-cred',  
  usernameVariable: 'GIT_USER',  
  passwordVariable: 'GIT_TOKEN'  
)]) {}
```

Rollback:

Việc rollback version cũ được thực hiện trong ArgoCD:

- Trong ArgoCD application, mở tab **History and Rollback**.
- Chọn 1 revision trước đó và nhấn **Rollback**.

1.3. Triển khai ứng dụng

Ứng dụng được triển khai bằng Helm chart thông qua ArgoCD, bao gồm:

- **Deployment**: Quản lý các pod dùng để chạy app
- **Service**: Cung cấp kết nối ổn định đến các pod.
- **Ingress**: Route traffic từ bên ngoài cluster vào Service.

- **HorizontalPodAutoscaler:** Tự động tăng/giảm (scale) số pod theo mức sử dụng CPU/Memory.

Lý do lựa chọn Helm:

- Dễ cài đặt và quản lý app cùng các resource liên quan (deployment, service, vv...)
- Dễ tùy biến các cấu hình trong app (số lượng replica, loại service, port sử dụng) với Helm template và file values.yaml.

Lý do lựa chọn ArgoCD:

- Với mô hình GitOps, mọi thay đổi về helm chart hay manifest trên repo sẽ được tự động đồng bộ xuống Kubernetes.
- Argo liên tục theo dõi tình trạng hiện tại của app và GitOps repo, và có thể tự động phát hiện drift và điều chỉnh ngay.
- Có sẵn tính năng rollback cho app.
- Argo có giao diện web, trực quan và dễ dùng hơn kubectl hay helm CLI.

1.4. Hiện thực hóa

The screenshot displays a Jenkins pipeline with the following stages: Start, Checkout SCM, Checkout, Install dependencies, Unit test, Docker Build, Docker Push, and Update GitOps Repo. The 'Update GitOps Repo' step is currently selected and expanded, showing a shell script that performs the following actions:

```

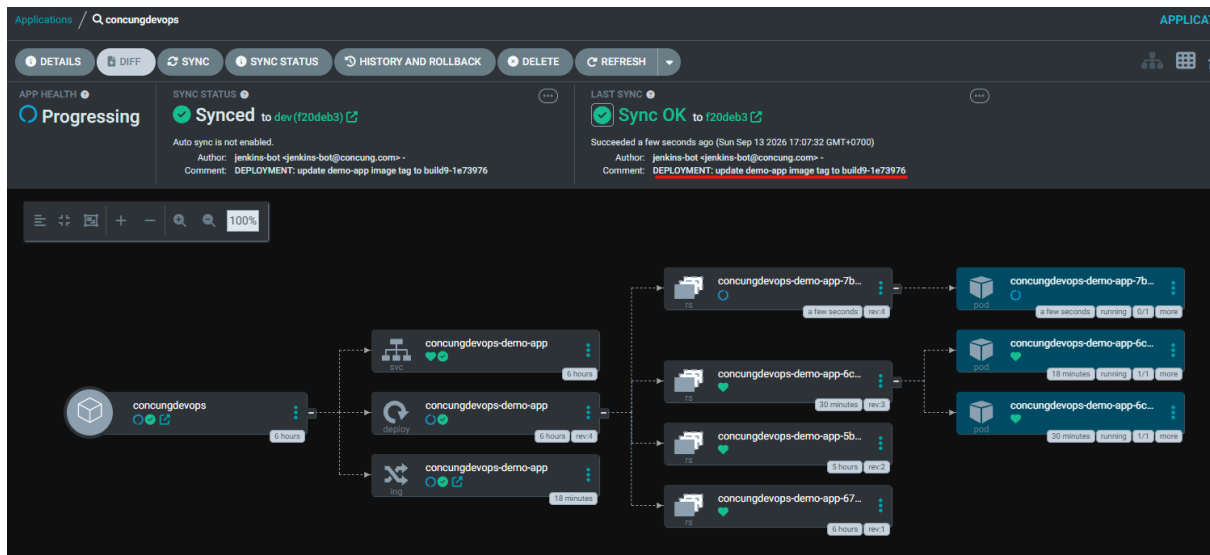
0 + git config --global user.email jenkins-bot@concung.com
1 + git config --global user.name jenkins-bot
2 + git clone https://****@github.com/****/devops-demo.git gitops-dir
3 Cloning into 'gitops-dir'...
4 + cd gitops-dir
5 + git checkout dev
6 branch 'dev' set up to track 'origin/dev'.
7 Switched to a new branch 'dev'
8 + sed -i 's/tag: .*/tag: "build9-1e73976"/' helm/demo-app/values.yaml
9 + git status --porcelain
10 + '[' -n 'M helm/demo-app/values.yaml' ]
11 + git add helm/demo-app/values.yaml
12 + git commit -m 'DEPLOYMENT: update demo-app image tag to build9-1e73976'
13 [dev f20deb3] DEPLOYMENT: update demo-app image tag to build9-1e73976
14 1 file changed, 1 insertion(+), 1 deletion(-)
15 + git push origin dev
16 To https://github.com/****/devops-demo.git
17 1e73976..f20deb3 dev -> dev
  
```

CI/CD pipeline chạy trên Jenkins

The screenshot shows the GitHub repository interface for 'devops-demo'. The current branch is 'dev', which is 2 commits ahead of and 1 commit behind the 'main' branch. The commit history is as follows:

Commit	Author	Message	Time
f20deb3	jenkins-bot	DEPLOYMENT: update demo-app image tag to build9-1e73976	1 minute ago
		helm/demo-app DEPLOYMENT: update demo-app image tag to build9-1e73976	1 minute ago
		jenkins Added GitOps deployment	4 hours ago
		test Refactored server.js	17 hours ago
		.dockerignore Added app src code, Dockerfile, Helm chart and pipeline	19 hours ago

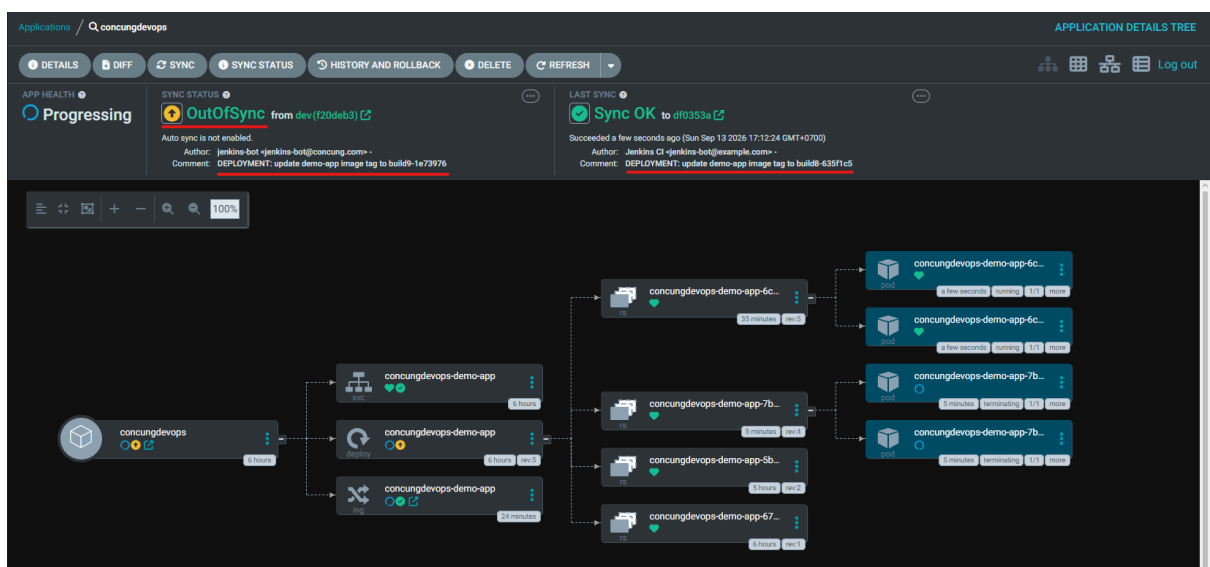
Commit được thực hiện để update image tag



ArgoCD cập nhật app lên version mới (build9-1e73976)

Details	Revision	Source Parameters
Deployed At: 33 minutes ago (Sun Sep 13 2026 16:37:58 GMT+0700) Time to deploy: 1s Initiated by: admin Active for: 11m10s	Revision: df0353a Authored by: Jenkins CI <jenkins-bot@example.com> 36 minutes ago (Sun Sep 13 2026 16:35:32 GMT+0700) DEPLOYMENT: update demo-app image tag to build8-635f1c5 GPG signature: -	Source Parameters URL: https://github.com/TonyQ2k3/devops-demo
Deployed At: 5 hours ago (Sun Sep 13 2026 12:26:55 GMT+0700) Time to deploy: 0s Initiated by: automated sync policy Active for: 4h11m	Revision: ea370f0 Authored by: Jenkins CI <jenkins-bot@example.com> 5 hours ago (Sun Sep 13 2026 12:25:44 GMT+0700) DEPLOYMENT: update demo-app image tag to build7-635f1c5 GPG signature: -	Source Parameters URL: https://github.com/TonyQ2k3/devops-demo

Danh sách các revision trước đó của app



Rollback về revision gần nhất (build8-635f1c5)

2. Task 3: Monitoring & Observability

2.1. Thiết kế hệ thống giám sát

Hệ thống giám sát sử dụng 4 tool là Prometheus, Loki, Alloy & Grafana, được cài đặt trong cùng cluster.

- Prometheus: Trích xuất và thu thập metrics từ các service chạy trong cluster.
- Loki & Alloy: Trích xuất log từ các service chạy trong cluster.
- Grafana: Giao diện web để tạo dashboard và truy vấn metrics/log

Hệ thống có 3 Dashboard:

- **Jenkins Dashboard:** Giám sát Jenkins controller và tình trạng job. Metrics được lấy từ plugin [Prometheus metrics](#) của Jenkins.
- **Demo App Dashboard:** Giám sát tình trạng ứng dụng web demo.
- **System Dashboard:** Giám sát tình trạng cluster và host.

Về alert, có 2 alert rule để phục vụ việc demo:

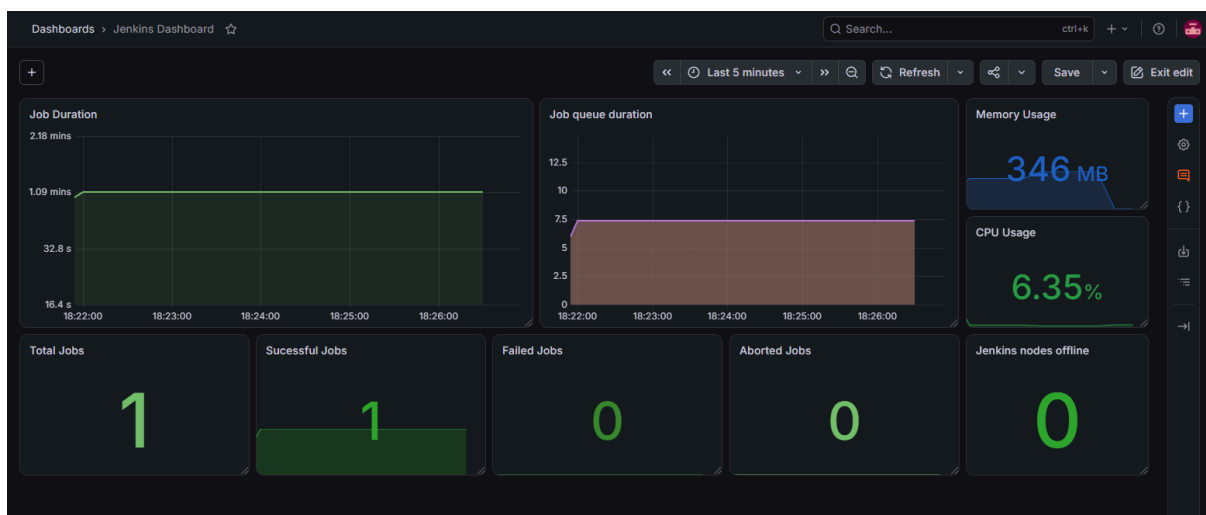
- **Jenkins Build Failed:** Kích hoạt khi có pipeline bị fail.
- **Demo App Failed:** Kích hoạt khi các pod chạy ứng dụng web bị down.

2.2. Cấu hình dashboard

2.2.1. Jenkins Dashboard

Các metrics quan trọng:

- Thời gian chạy job trung bình
- Thời gian job queue (chờ chạy)
- Mức sử dụng CPU/Memory
- Tổng số lượng job success/fail



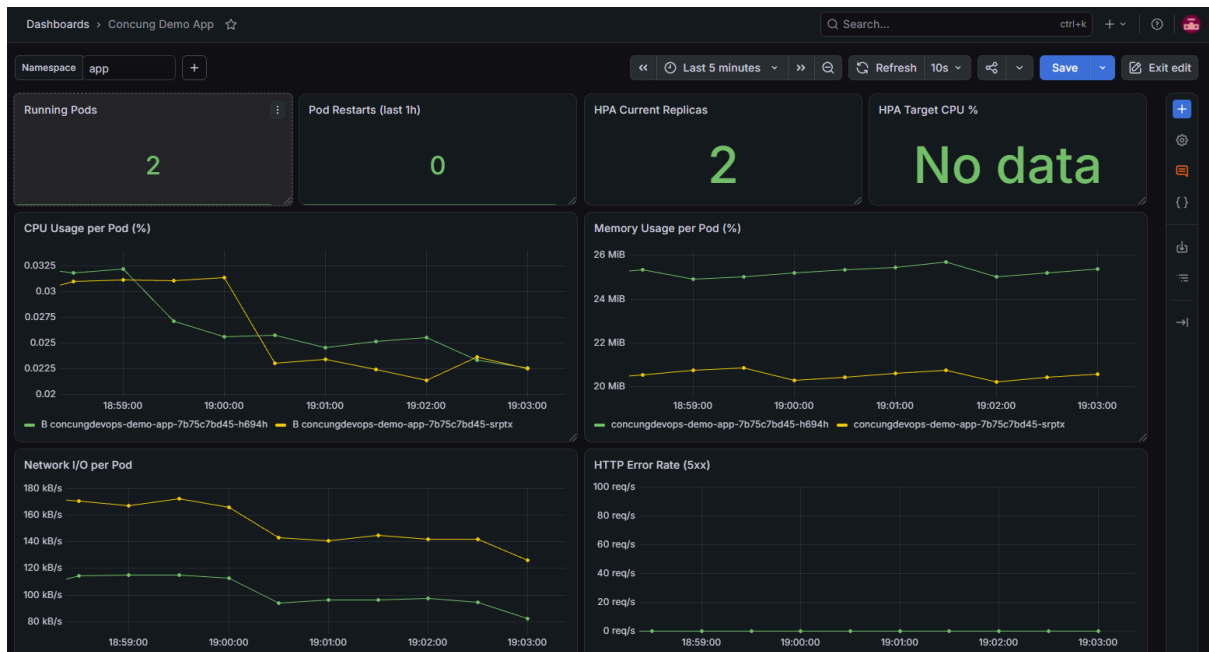
Jenkins dashboard

2.2.2. Demo App Dashboard

Các metrics quan trọng:

- Số pod app đang chạy

- Số lần pod restart
- Mức sử dụng CPU/Memory
- Lưu lượng mạng app đang xử lý
- Lượng request bị fail (5xx error rate)

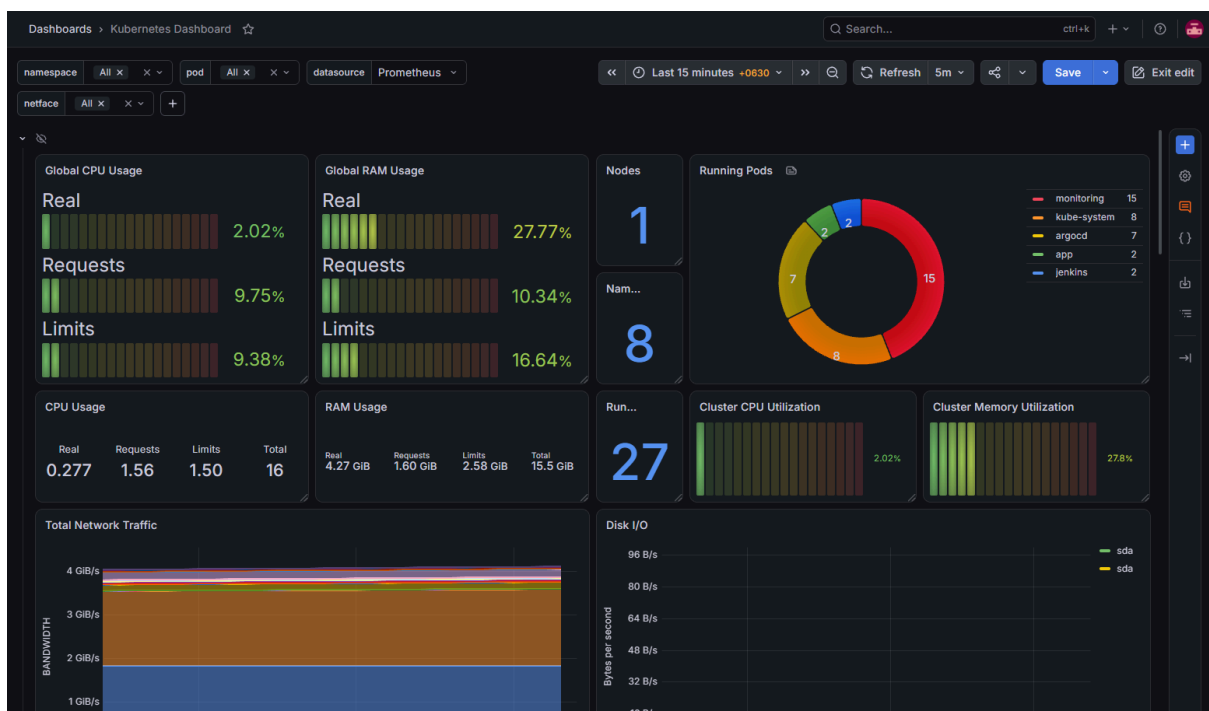


Demo App dashboard

2.2.3. System Dashboard

Các metrics quan trọng:

- Tổng mức sử dụng CPU/Memory của tất cả app đang chạy (usage/request/limit).
- Mức sử dụng CPU/Memory của cluster/host.
- Tổng lưu lượng mạng đang đi qua cluster/host.
- Tổng lưu lượng I/O thực hiện trên ổ cứng.

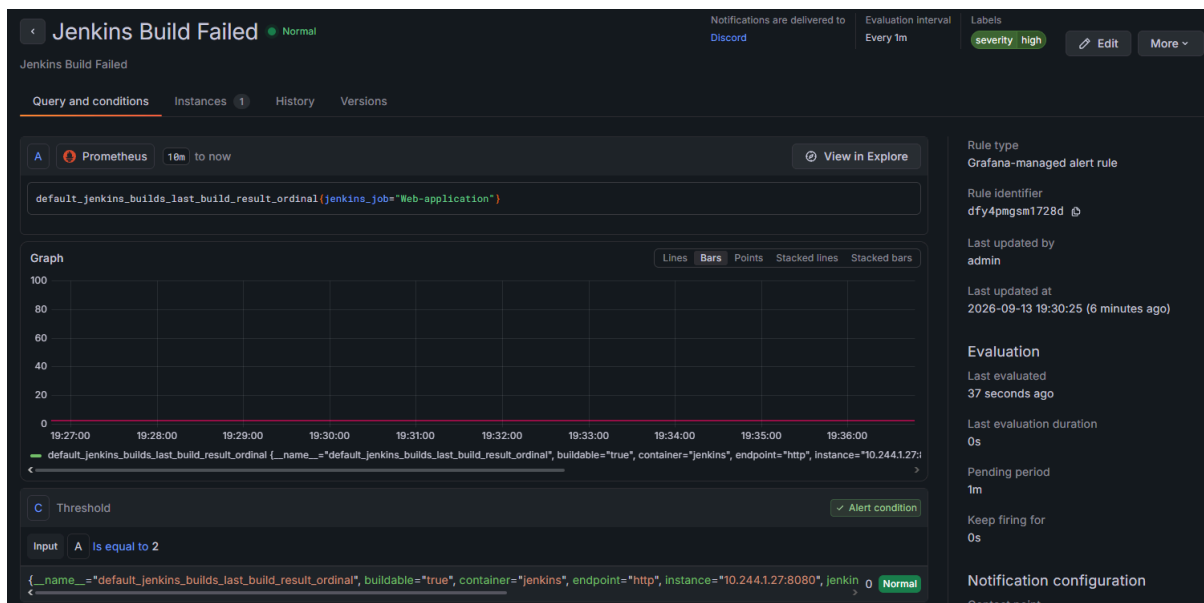


2.3. Cấu hình alert

2.3.1. Jenkins Build Failed

Alert được kích hoạt khi pipeline gần nhất trả về kết quả fail. Trong Prometheus metrics, `default_jenkins_builds_last_build_result_ordinal` trả về '2' khi job bị fail.

```
default_jenkins_builds_last_build_result_ordinal{jenkins_job="Web-application"} == 2
```

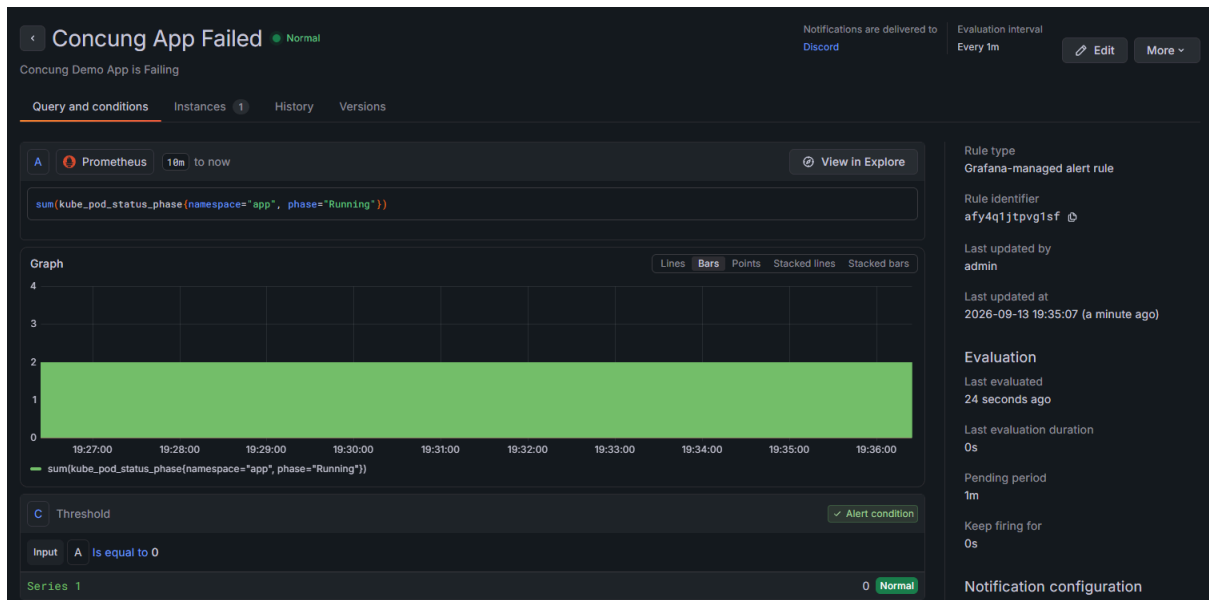


Jenkins alert

2.3.2. Demo App Failed

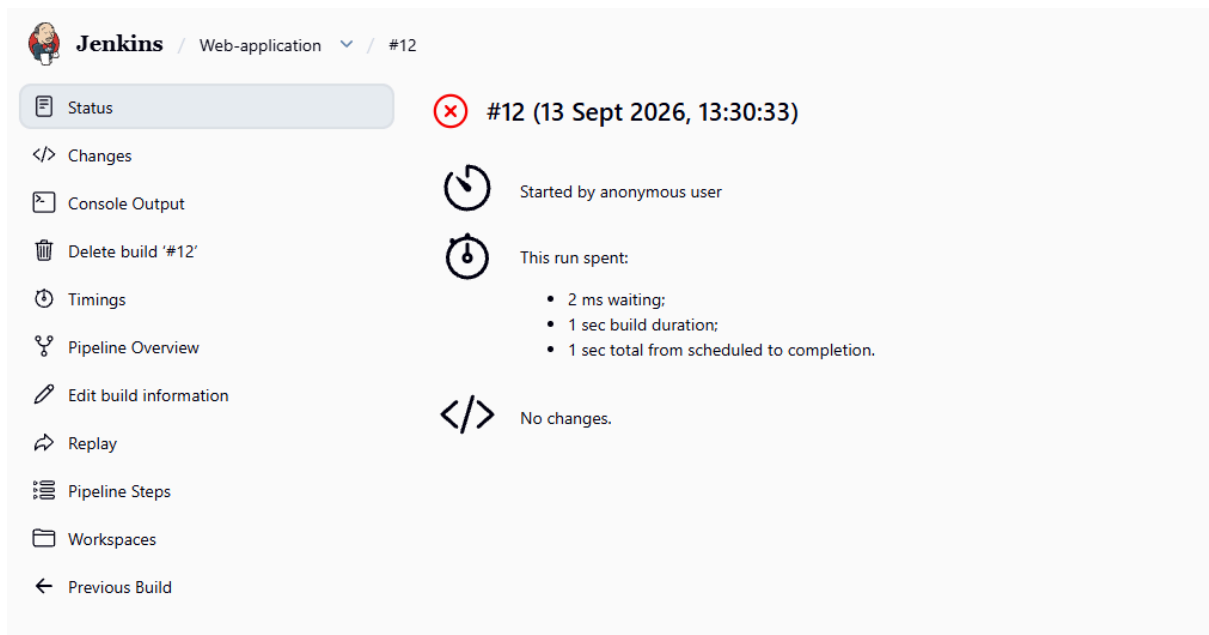
Alert được kích hoạt khi tất cả pod bị down.

```
sum(kube_pod_status_phase{namespace="app", phase="Running"}) == 0
```

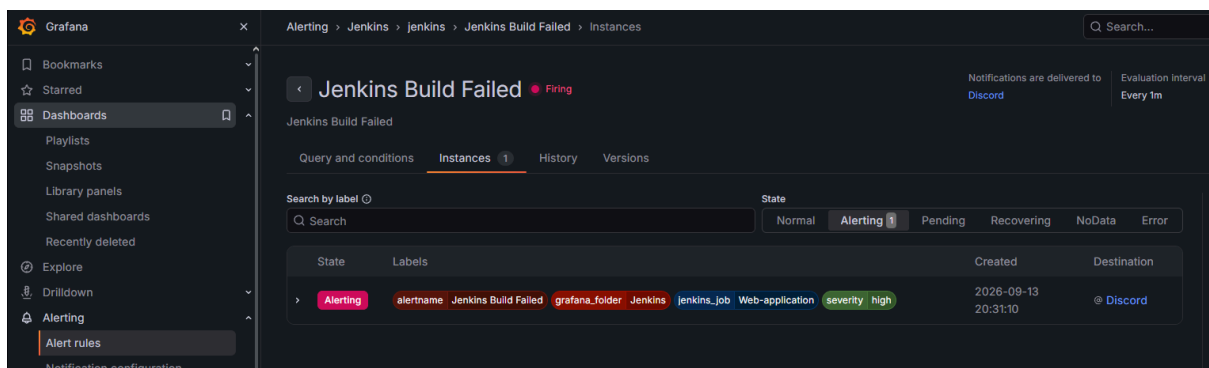


Demo App alert

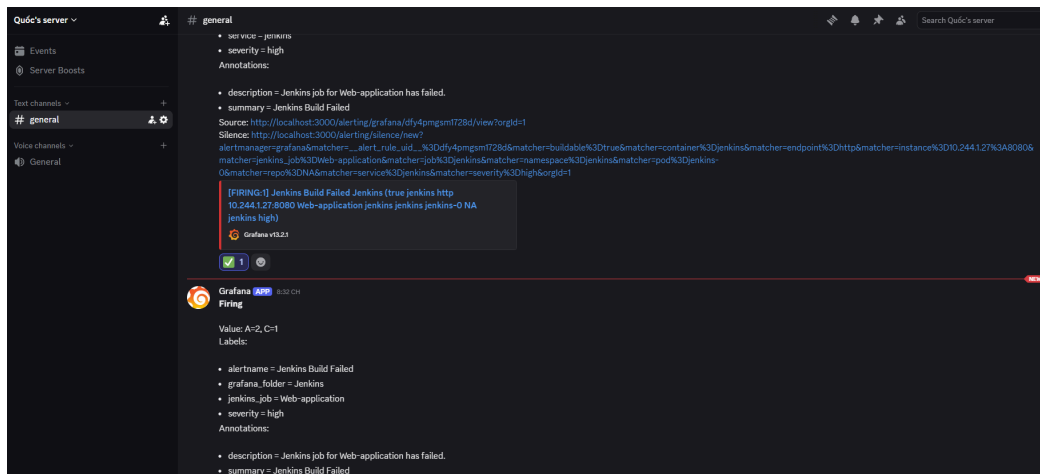
2.4. Hiện thực hóa



Pipeline bị fail trong Jenkins



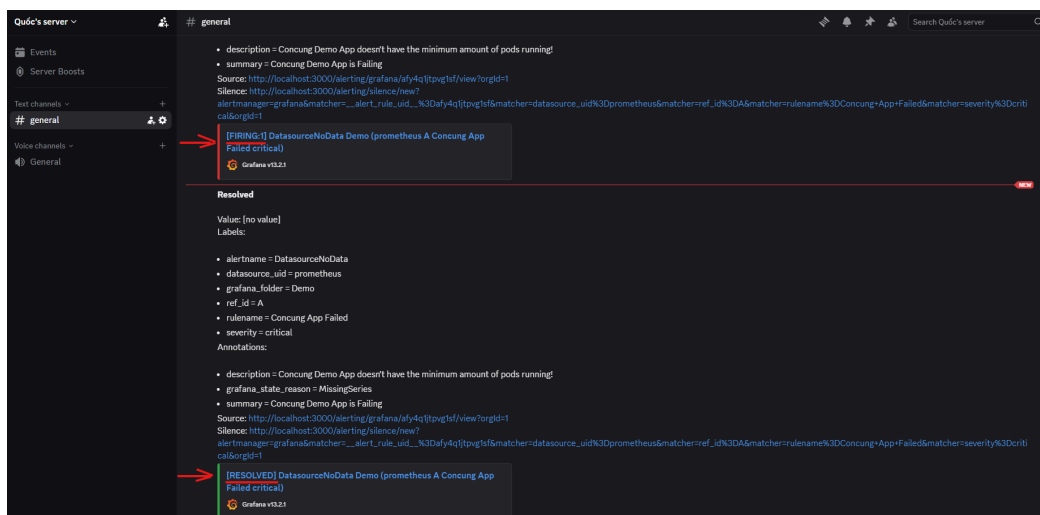
Jenkins alert kích hoạt



Alert được gửi trong nhóm chat developer

```
PS D:\Projects\devops-demo> kubectl get deployment -n app
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
concurringdevops-demo-app          2/2     2             2           10h
PS D:\Projects\devops-demo> kubectl delete deployment --all -n app
deployment.apps "concurringdevops-demo-app" deleted from app namespace
PS D:\Projects\devops-demo> kubectl delete deployment --all -n app
No resources found
PS D:\Projects\devops-demo> kubectl delete deployment --all -n app
No resources found
```

Xóa app deployment để làm mất pod



Alert demo app trước và sau khi được ArgoCD re-deploy

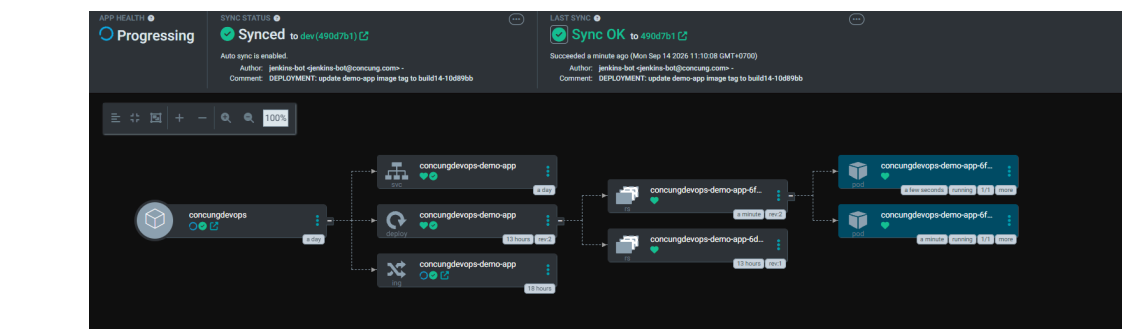
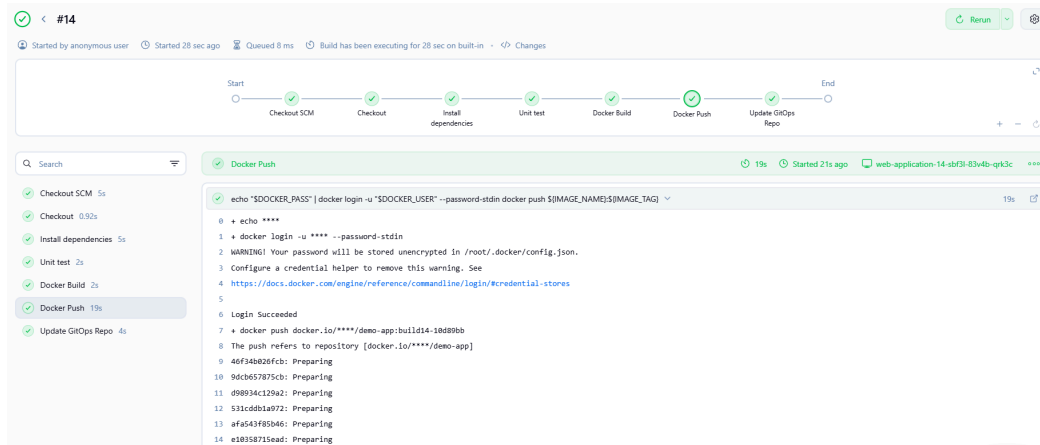
3. Task 5: Incident Response

3.1. Mô phỏng sự cố

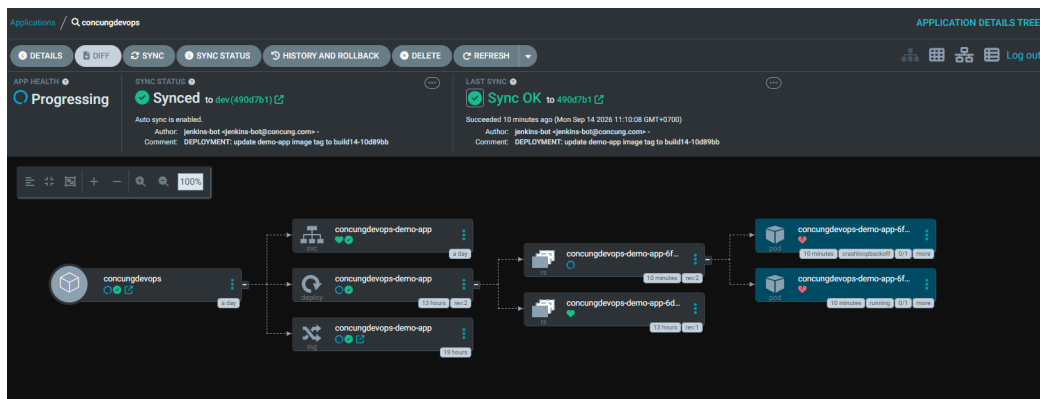
Trong ví dụ này, ta sẽ thêm một đoạn code để khiến app bị crash khi người dùng truy cập vào path `/crash` (mô phỏng 1 đoạn code lỗi).

```
app.get('/crash', (req, res) => {
  res.status(500).send('App is failing...');
  console.error('This is a code error!');
  process.exit(1);
});
```

Sau đó, commit lên Git và chạy pipeline để deploy đoạn code:

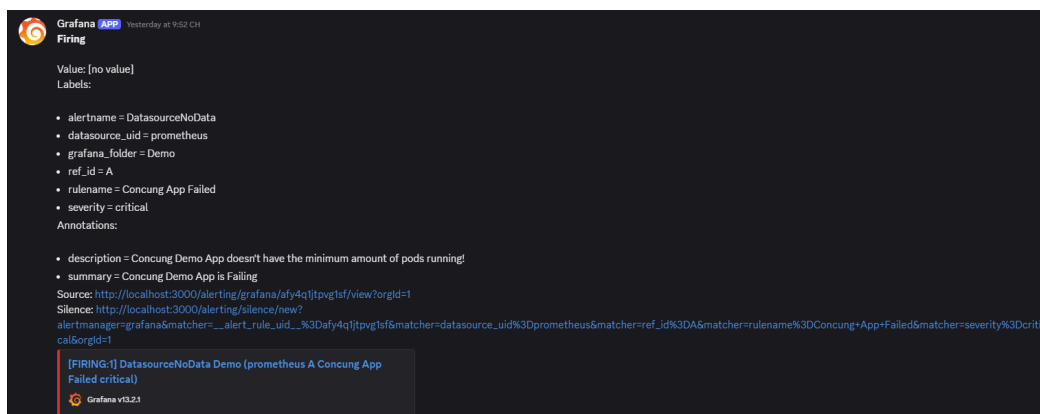


Khi người dùng truy cập vào phần app chứa đoạn code lỗi, app sẽ bị crash và ứng dụng web sẽ bị down.

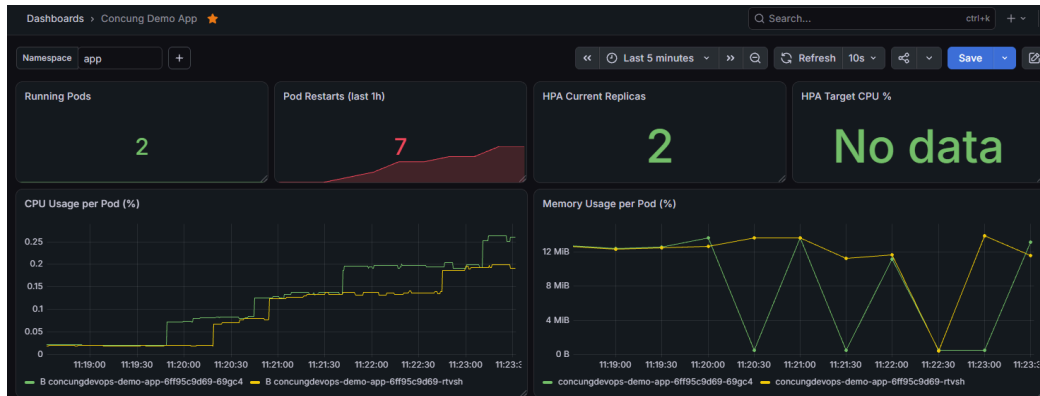


3.2. Phản ứng với sự cố

Khi app bị crash, Grafana alert sẽ kích hoạt và gửi tin nhắn lên group chat.

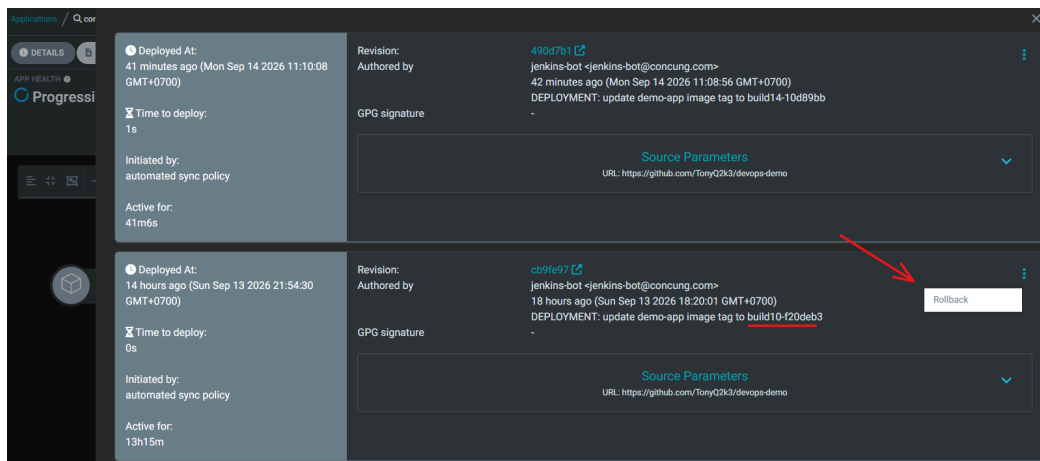


Grafana Alert được gửi lên kênh Discord



Sự cố được phản ánh trên Demo App Dashboard

Vì sự cố xảy ra ngay sau khi deploy version mới, phản ứng tức thời sẽ là rollback lại version cũ, và liên tục theo dõi xem app có hoạt động lại không.



Rollback về phiên bản trước đó

Khi xác minh lỗi không còn xuất hiện, ta sẽ bắt đầu đi tìm nguyên nhân.

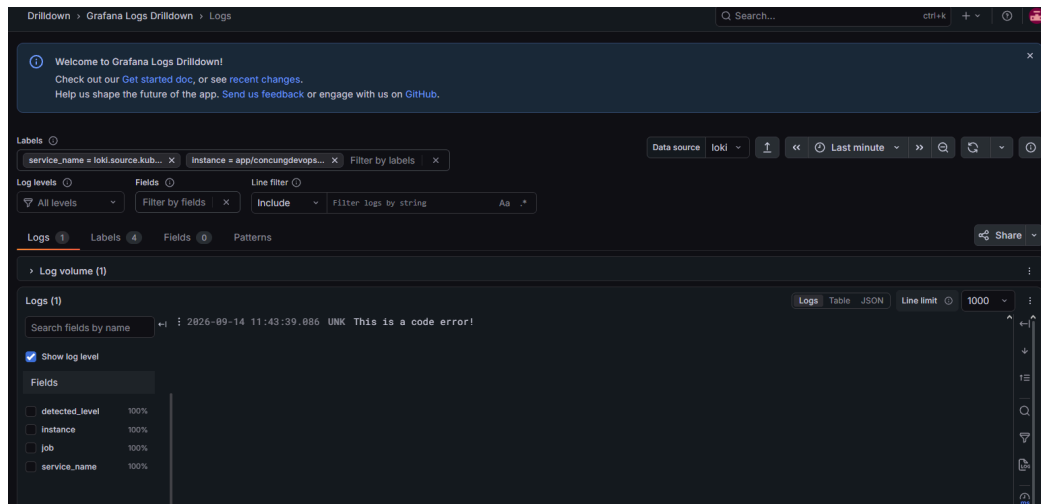
3.3. Xác định nguyên nhân

Có nhiều nguyên nhân khiến user không truy cập được app, nhưng với hệ thống giám sát, developer có thể nhanh chóng khoanh vùng và xác định root cause:

- Alert chỉ ra các pod đang bị down (Prometheus alert)
- Số lần restart pod tăng liên tục (Grafana dashboard)
- Status pod hiển thị CrashLoopBackOff (ArgoCD dashboard)

Developer có thể dễ dàng nhận ra nguyên nhân nằm ở việc pod bị CrashLoopBackOff, và lỗi nằm ở code app hoặc lỗi cấu hình (misconfiguration), không phải do scheduling hay OOMKill.

Để xác định chính xác root cause, developer tiếp tục check log của pod. Vì hệ thống dùng Loki để lưu log tập trung, developer có thể truy xuất log kể cả khi pod đã bị xóa (sau khi rollback).



Log demo app được quản lý bởi Loki

Lúc này developer có thể xác định được root cause là do source code, và có thể tiến hành khắc phục.

3.4. Phòng tránh sự cố tương tự

Việc code bị lỗi vẫn được deploy chứng tỏ unit test được viết chưa cẩn thận, dẫn đến không phát hiện được bug. Để phòng tránh những sự cố tương tự trong tương lai, ta có thể chỉnh code local và trong pipeline để thêm bước thống kê độ bao phủ unit test (VD: `npm test -- --coverage`).

Việc thêm bước kiểm tra test coverage có thể giúp developer phát hiện những trường hợp chưa có unit test hoặc test chưa đầy đủ, giúp hạn chế nguy cơ code lỗi bị deploy.